

Topic: Merge and Groupby

Converted from .ipynb

Import libs

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Download datasets

In [2]:

```
!curl -O https://raw.githubusercontent.com/jakevdp/data-USstates/master/state-population.csv
!curl -O https://raw.githubusercontent.com/jakevdp/data-USstates/master/state-areas.csv
!curl -O https://raw.githubusercontent.com/jakevdp/data-USstates/master/state-abbrevs.csv
```

[output]

% Total		% Received		% Xferd		Average Speed		Time	Time	Time	Current
						Dload	Upload	Total	Spent	Left	Speed
100	57935	100	57935	0	0	229k	0	--:--:--	--:--:--	--:--:--	229k
% Total		% Received		% Xferd		Average Speed		Time	Time	Time	Current
						Dload	Upload	Total	Spent	Left	Speed
100	835	100	835	0	0	3806	0	--:--:--	--:--:--	--:--:--	3812
% Total		% Received		% Xferd		Average Speed		Time	Time	Time	Current
						Dload	Upload	Total	Spent	Left	Speed
100	872	100	872	0	0	4782	0	--:--:--	--:--:--	--:--:--	4765

Define utilities

In [3]:

```

class display(object):
    """Display HTML representation of multiple objects"""
    table_template = \
    """
    <div style="float: left; padding: 1px;">
    <p style='font-family:"Courier New", Courier, monospace;
    color:olive;
    font-size:120%;
    font-weight:bold;'>{0}</p>
    <hr>
    {1}
    </div>
    """
    caption_template = """\
    <div style='
    background-color:#D6DBDF;'>
    <p style='font-family:"Courier New", Courier, monospace;
    color:#0000FF;
    font-size:120%;
    font-weight:bold;'>
    {0}
    </p>
    </div>
    """
    def __init__(self, *args):
        self.frames = list(filter(lambda item: not item.strip().startswith('#'), args))
        self.caption = list(filter(lambda item: item.strip().startswith('#'), args))
        self.caption = [item[1:].strip().upper() for item in self.caption]
        if len(self.caption) == 0:
            self.caption = None
        else:
            self.caption = ':'.join(self.caption)

    def _repr_html_(self):
        if self.caption is not None:
            return self.caption_template.format(self.caption) + \
                '\n'.join(self.table_template.format(frame, eval(frame)._repr_html_())
                           for frame in self.frames)
        else:
            return '\n'.join(self.table_template.format(frame, eval(frame)._repr_html_())
                              for frame in self.frames)

    #def __repr__(self):
    #    return '\n\n'.join(a + '\n' + repr(eval(a))
    #                        for a in self.args)

```

One-to-One

In [4]:

```

df1 = pd.DataFrame({'employee': ['Bob', 'Jake', 'Lisa', 'Sue', 'Mary'],
                    'group': ['Accounting', 'Engineering', 'Engineering', 'HR', 'Engineering']})
df2 = pd.DataFrame({'employee': ['Lisa', 'Bob', 'Jake', 'Sue', 'Tom', 'Anne'],
                    'hire_date': [2004, 2008, 2012, 2014, 2010, 2008]})
display('df1', 'df2')

```

[output]

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR
4	Mary	Engineering
	employee	hire_date
0	Lisa	2004
1	Bob	2008
2	Jake	2012
3	Sue	2014
4	Tom	2010
5	Anne	2008

inner - default

In [5]:

```
pd.merge(df1, df2)
```

[output]

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

inner - explicit

In [6]:

```
pd.merge(df1, df2, how="inner")
```

[output]

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

on vs left_on and right_on

In [7]:

```
df1 = pd.DataFrame({'employeeA': ['Bob', 'Jake', 'Lisa', 'Sue', 'Mary'],
                    'group': ['Accounting', 'Engineering', 'Engineering', 'HR', 'Engineering']})
df2 = pd.DataFrame({'employeeB': ['Lisa', 'Bob', 'Jake', 'Sue', 'Tom', 'Anne'],
                    'hire_date': [2004, 2008, 2012, 2014, 2010, 2008]})
display('df1', 'df2')
```

[output]

	employeeA	group
0	Bob	Accounting

	employeeA	group
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR
4	Mary	Engineering
	employeeB	hire_date
0	Lisa	2004
1	Bob	2008
2	Jake	2012
3	Sue	2014
4	Tom	2010
5	Anne	2008

In [8]:

```
pd.merge(df1, df2, how="inner", left_on="employeeA", right_on="employeeB").drop(["employeeB"], axis=1)
```

[output]

	employeeA	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

validate the relationship

In [9]:

```
df1 = pd.DataFrame({'employee': ['Bob', 'Jake', 'Lisa', 'Sue', 'Mary'],
                    'group': ['Accounting', 'Engineering', 'Engineering', 'HR', 'Engineering']})
df2 = pd.DataFrame({'employee': ['Lisa', 'Bob', 'Jake', 'Sue', 'Tom', 'Anne'],
                    'hire_date': [2004, 2008, 2012, 2014, 2010, 2008]})
display('df1', 'df2')
```

[output]

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR
4	Mary	Engineering
	employee	hire_date
0	Lisa	2004
1	Bob	2008
2	Jake	2012
3	Sue	2014
4	Tom	2010
5	Anne	2008

In [10]:

```
pd.merge(df1, df2, how="inner", validate="one_to_one")
```

[output]

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

In [11]:

```
df1 = pd.DataFrame({'employee': ['Bob', 'Jake', 'Lisa', 'Sue', 'Mary'],  
                    'group': ['Accounting', 'Engineering', 'Engineering', 'HR', 'Engineering']})  
df3 = pd.DataFrame({'employee': ['Lisa', 'Bob', 'Jake', 'Sue', 'Tom', 'Anne', 'Bob'],  
                    'hire_date': [2004, 2008, 2012, 2014, 2010, 2008, 2026]})  
display('df1', 'df3')
```

[output]

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR
4	Mary	Engineering
	employee	hire_date
0	Lisa	2004
1	Bob	2008
2	Jake	2012
3	Sue	2014
4	Tom	2010
5	Anne	2008
6	Bob	2026

In [12]:

```
try:  
    df = pd.merge(df1, df2, how="inner", validate="one_to_one")  
    print(df)  
  
except Exception as e:  
    print(e)
```

[output]

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

In [13]:

```
try:  
    df = pd.merge(df1, df3, how="inner", validate="one_to_one")  
    print(df)  
  
except Exception as e:  
    print(e)
```

[output]

Merge keys are not unique in right dataset; not a one-to-one merge

inner => left and right

In [14]:

```
df1 = pd.DataFrame({'employee': ['Bob', 'Jake', 'Lisa', 'Sue', 'Mary'],
                    'group': ['Accounting', 'Engineering', 'Engineering', 'HR', 'Engineering']})
df2 = pd.DataFrame({'employee': ['Lisa', 'Bob', 'Jake', 'Sue', 'Tom', 'Anne'],
                    'hire_date': [2004, 2008, 2012, 2014, 2010, 2008]})
display(df1, df2)
```

[output]

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR
4	Mary	Engineering
	employee	hire_date
0	Lisa	2004
1	Bob	2008
2	Jake	2012
3	Sue	2014
4	Tom	2010
5	Anne	2008

In [15]:

```
df = pd.merge(df1, df2, how="left")
print(df)
```

[output]

	employee	group	hire_date
0	Bob	Accounting	2008.0
1	Jake	Engineering	2012.0
2	Lisa	Engineering	2004.0
3	Sue	HR	2014.0
4	Mary	Engineering	NaN

In [16]:

```
df = pd.merge(df1, df2, how="right")
print(df)
```

[output]

	employee	group	hire_date
0	Lisa	Engineering	2004
1	Bob	Accounting	2008
2	Jake	Engineering	2012
3	Sue	HR	2014
4	Tom	NaN	2010
5	Anne	NaN	2008

cross

In [17]:

```
display("df1", "df2")
```

[output]

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR
4	Mary	Engineering
	employee	hire_date
0	Lisa	2004
1	Bob	2008
2	Jake	2012
3	Sue	2014
4	Tom	2010
5	Anne	2008

In [18]:

```
df3 = pd.merge(df1, df2, how='cross')  
display('df3')
```

[output]

	employee_x	group	employee_y	hire_date
0	Bob	Accounting	Lisa	2004
1	Bob	Accounting	Bob	2008
2	Bob	Accounting	Jake	2012
3	Bob	Accounting	Sue	2014
4	Bob	Accounting	Tom	2010
5	Bob	Accounting	Anne	2008
6	Jake	Engineering	Lisa	2004
7	Jake	Engineering	Bob	2008
8	Jake	Engineering	Jake	2012
9	Jake	Engineering	Sue	2014
10	Jake	Engineering	Tom	2010
11	Jake	Engineering	Anne	2008
12	Lisa	Engineering	Lisa	2004
13	Lisa	Engineering	Bob	2008
14	Lisa	Engineering	Jake	2012
15	Lisa	Engineering	Sue	2014
16	Lisa	Engineering	Tom	2010
17	Lisa	Engineering	Anne	2008
18	Sue	HR	Lisa	2004
19	Sue	HR	Bob	2008
20	Sue	HR	Jake	2012
21	Sue	HR	Sue	2014
22	Sue	HR	Tom	2010
23	Sue	HR	Anne	2008

	employee_x	group	employee_y	hire_date
24	Mary	Engineering	Lisa	2004
25	Mary	Engineering	Bob	2008
26	Mary	Engineering	Jake	2012
27	Mary	Engineering	Sue	2014
28	Mary	Engineering	Tom	2010
29	Mary	Engineering	Anne	2008

one-to-many

In [19]:

```
df3 = pd.merge(df1, df2, how='inner')
df4 = pd.DataFrame({'group': ['Accounting', 'Engineering', 'HR', 'Data Analytics'],
                    'supervisor': ['Carly', 'Guido', 'Steve', 'David']})
display('df3', 'df4')
```

[output]

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014
	group	supervisor	
0	Accounting	Carly	
1	Engineering	Guido	
2	HR	Steve	
3	Data Analytics	David	

In [20]:

```
pd.merge(df3, df4)
```

[output]

	employee	group	hire_date	supervisor
0	Bob	Accounting	2008	Carly
1	Jake	Engineering	2012	Guido
2	Lisa	Engineering	2004	Guido
3	Sue	HR	2014	Steve

In [21]:

```
pd.merge(df3, df4)
```

[output]

	employee	group	hire_date	supervisor
0	Bob	Accounting	2008	Carly
1	Jake	Engineering	2012	Guido
2	Lisa	Engineering	2004	Guido
3	Sue	HR	2014	Steve

In [22]:

```
pd.merge(df3, df4, validate="many_to_one")
```

[output]

	employee	group	hire_date	supervisor
0	Bob	Accounting	2008	Carly
1	Jake	Engineering	2012	Guido
2	Lisa	Engineering	2004	Guido
3	Sue	HR	2014	Steve

Grouping

In [23]:

```
students = [  
    'An',  
    'Mai',  
    'Loan',  
    'Tien'  
] #assumption: name is also code  
courses = [  
    "Algorithms",  
    "Computer Architecture",  
    "Programming"  
]  
  
st_frame = pd.DataFrame(students, columns=['Student'])  
co_frame = pd.DataFrame(courses, columns=['Course'])  
scores = pd.merge(st_frame, co_frame, how='cross')  
display('st_frame', 'co_frame', 'scores')
```

[output]

	Student	
0	An	
1	Mai	
2	Loan	
3	Tien	
	Course	
0	Algorithms	
1	Computer Architecture	
2	Programming	
	Student	Course
0	An	Algorithms
1	An	Computer Architecture
2	An	Programming
3	Mai	Algorithms
4	Mai	Computer Architecture
5	Mai	Programming
6	Loan	Algorithms
7	Loan	Computer Architecture
8	Loan	Programming
9	Tien	Algorithms
10	Tien	Computer Architecture

	Student	
11	Tien	Programming

In [24]:

```
students = [
    'An',
    'Mai',
    'Loan',
    'Tien'
] #assumption: name is also code
courses = [
    "Algorithms",
    "Computer Architecture",
    "Programming"
]
st_frame = pd.DataFrame(students, columns=['Student'])
co_frame = pd.DataFrame(courses, columns=['Course'])
scores = pd.merge(st_frame, co_frame, how='cross')
scores['Score'] = pd.Series(data=np.random.randint(3, 11, (scores.shape[0])))
display('st_frame', 'co_frame', 'scores')
```

[output]

	Student		
0	An		
1	Mai		
2	Loan		
3	Tien		
	Course		
0	Algorithms		
1	Computer Architecture		
2	Programming		
	Student	Course	Score
0	An	Algorithms	3
1	An	Computer Architecture	4
2	An	Programming	5
3	Mai	Algorithms	3
4	Mai	Computer Architecture	8
5	Mai	Programming	7
6	Loan	Algorithms	4
7	Loan	Computer Architecture	8
8	Loan	Programming	10
9	Tien	Algorithms	8
10	Tien	Computer Architecture	8
11	Tien	Programming	7

In [25]:

```
display("scores")
```

[output]

	Student	Course	Score
0	An	Algorithms	3

	Student	Course	Score
1	An	Computer Architecture	4
2	An	Programming	5
3	Mai	Algorithms	3
4	Mai	Computer Architecture	8
5	Mai	Programming	7
6	Loan	Algorithms	4
7	Loan	Computer Architecture	8
8	Loan	Programming	10
9	Tien	Algorithms	8
10	Tien	Computer Architecture	8
11	Tien	Programming	7

aggregation

In [26]:

```
scores.groupby("Student").agg({"Score": ["min", "max", np.mean]}).reset_index()
```

[output]

/tmp/ipykernel_1659/2328954905.py:1: FutureWarning: The provided callable <function mean at 0x7f6fd7d35800> is current
 scores.groupby("Student").agg({"Score": ["min", "max", np.mean]}).reset_index()

[output]

	Student	Score		
		min	max	mean
0	An	3	5	4.000000
1	Loan	4	10	7.333333
2	Mai	3	8	6.000000
3	Tien	7	8	7.666667

In [27]:

```
scores.groupby("Student").agg({
    "Score": ["min", "max", "mean"]
}).reset_index()
```

[output]

	Student	Score		
		min	max	mean
0	An	3	5	4.000000
1	Loan	4	10	7.333333
2	Mai	3	8	6.000000
3	Tien	7	8	7.666667

In [28]:

```
scores.groupby("Student").agg({
    "Score": ["min", "max", "mean"],
    "Course": "count"
}).reset_index()
```

[output]

	Student	Score	Course		
		min	max	mean	count

	Student	Score	Course		
0	An	3	5	4.000000	3
1	Loan	4	10	7.333333	3
2	Mai	3	8	6.000000	3
3	Tien	7	8	7.666667	3

groupby: principle

* steps:

1. split => groups
2. stats for each group
3. combine results

In [29]:

```
# SPLIT
for (st, group) in scores.groupby('Student'):
    print("{0:10s} shape={1}".format(st, group.shape))
```

[output]

```
An          shape=(3, 3)
Loan        shape=(3, 3)
Mai         shape=(3, 3)
Tien        shape=(3, 3)
```

In [30]:

```
type(scores.groupby('Student'))
```

[output]

```
pandas.core.groupby.generic.DataFrameGroupBy
```

In [31]:

```
# SPLIT
for (st, group) in scores.groupby('Student'):
    print("grouping value: ");
    print(st)
    print()

    print("data frame: ")
    print("type(group):")
    print(type(group))
    print()

    print("group:")
    print(group)
    print()

#
```

[output]

```

grouping value:
An

data frame:
type(group):
<class 'pandas.core.frame.DataFrame'>

group:
  Student      Course  Score
0      An      Algorithms    3
1      An  Computer Architecture    4
2      An      Programming    5

grouping value:
Loan

data frame:
type(group):
<class 'pandas.core.frame.DataFrame'>

group:
  Student      Course  Score
6      Loan      Algorithms    4
7      Loan  Computer Architecture    8
8      Loan      Programming   10

grouping value:
Mai

data frame:
type(group):
<class 'pandas.core.frame.DataFrame'>

group:
  Student      Course  Score
3      Mai      Algorithms    3
4      Mai  Computer Architecture    8
5      Mai      Programming    7

grouping value:
Tien

data frame:
type(group):
<class 'pandas.core.frame.DataFrame'>

group:
  Student      Course  Score
9      Tien      Algorithms    8
10     Tien  Computer Architecture    8
11     Tien      Programming    7

```

filter

In [32]:

```

df = scores.groupby("Student").filter(lambda subframe: subframe['Score'].mean() >= 5.0)
display('df')

```

[output]

	Student	Course	Score
3	Mai	Algorithms	3
4	Mai	Computer Architecture	8

	Student	Course	Score
5	Mai	Programming	7
6	Loan	Algorithms	4
7	Loan	Computer Architecture	8
8	Loan	Programming	10
9	Tien	Algorithms	8
10	Tien	Computer Architecture	8
11	Tien	Programming	7

In [33]:

```
f = scores.groupby("Student").filter(lambda subframe: subframe['Score'].mean() < 5.0)
display('df')
```

[output]

	Student	Course	Score
0	An	Algorithms	3
1	An	Computer Architecture	4
2	An	Programming	5

In [34]:

```
def filtered_by(df):
    print(type(df))
    print()
    return df['Score'].mean() < 5.0

df = scores.groupby("Student").filter(filtered_by)
display('df')
```

[output]

```
<class 'pandas.core.frame.DataFrame'>

<class 'pandas.core.frame.DataFrame'>

<class 'pandas.core.frame.DataFrame'>

<class 'pandas.core.frame.DataFrame'>
```

[output]

	Student	Course	Score
0	An	Algorithms	3
1	An	Computer Architecture	4
2	An	Programming	5

transformation

In [35]:

```
df_new = scores.copy()
df_new['New-Score'] = scores.groupby('Student')['Score'].transform(lambda x: x + x.mean())
display('df_new')
```

[output]

	Student	Course	Score	New-Score
0	An	Algorithms	3	7.000000
1	An	Computer Architecture	4	8.000000

	Student	Course	Score	New-Score
2	An	Programming	5	9.000000
3	Mai	Algorithms	3	9.000000
4	Mai	Computer Architecture	8	14.000000
5	Mai	Programming	7	13.000000
6	Loan	Algorithms	4	11.333333
7	Loan	Computer Architecture	8	15.333333
8	Loan	Programming	10	17.333333
9	Tien	Algorithms	8	15.666667
10	Tien	Computer Architecture	8	15.666667
11	Tien	Programming	7	14.666667

In [36]:

```
df_new = scores.copy()

def transformed_func(data):
    print("type(data)")
    print(type(data))
    print()
    return data + data.mean()

df_new['New-Score'] = scores.groupby('Student')['Score'].transform(transformed_func)
display('df_new')
```

[output]

```
type(data)
<class 'pandas.core.series.Series'>

type(data)
<class 'pandas.core.series.Series'>

type(data)
<class 'pandas.core.series.Series'>

type(data)
<class 'pandas.core.series.Series'>
```

[output]

	Student	Course	Score	New-Score
0	An	Algorithms	3	7.000000
1	An	Computer Architecture	4	8.000000
2	An	Programming	5	9.000000
3	Mai	Algorithms	3	9.000000
4	Mai	Computer Architecture	8	14.000000
5	Mai	Programming	7	13.000000
6	Loan	Algorithms	4	11.333333
7	Loan	Computer Architecture	8	15.333333
8	Loan	Programming	10	17.333333
9	Tien	Algorithms	8	15.666667
10	Tien	Computer Architecture	8	15.666667
11	Tien	Programming	7	14.666667

apply

In [37]:

```
def add_bonus(frame):
    frame['Added'] = frame['Score'] + frame['Score'].mean()
    return frame

df_new = scores.groupby('Student').apply(add_bonus)
display('df_new')
```

[output]

```
/tmp/ipykernel_1659/4170740569.py:5: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This
df_new = scores.groupby('Student').apply(add_bonus)
```

[output]

		Student	Course	Score	Added
Student					
An	0	An	Algorithms	3	7.000000
1	An	Computer Architecture	4	8.000000	
2	An	Programming	5	9.000000	
Loan	6	Loan	Algorithms	4	11.333333
7	Loan	Computer Architecture	8	15.333333	
8	Loan	Programming	10	17.333333	
Mai	3	Mai	Algorithms	3	9.000000
4	Mai	Computer Architecture	8	14.000000	
5	Mai	Programming	7	13.000000	
Tien	9	Tien	Algorithms	8	15.666667
10	Tien	Computer Architecture	8	15.666667	
11	Tien	Programming	7	14.666667	

In [38]:

```
scores.groupby('Student')['Score']
```

[output]

```
<pandas.core.groupby.generic.SeriesGroupBy object at 0x7f6f97bdda30>
```

In [39]:

```
def add_bonus(data):
    return data + 1.0

s = scores.groupby('Student')['Score'].apply(add_bonus)
s
```

[output]

		Score
Student		
An	0	4.0
1	5.0	
2	6.0	
Loan	6	5.0
7	9.0	
8	11.0	
Mai	3	4.0

		Score
4	9.0	
5	8.0	
Tien	9	9.0
10	9.0	
11	8.0	

Viz